

## ✓ LABORATORIO N. 09

### Semana 9 - Administracion y Almacenamiento de Base de Datos

**Curso:** Fundamentos de Gestion de Datos

**Herramientas:** Google Colab, SQLite, Python, pandas

**Base real:** SQLite descargada/subida desde GitHub

**Repositorio:** [https://github.com/Rociosayan/PMD1\\_Fundamentos\\_Gestion\\_Datos](https://github.com/Rociosayan/PMD1_Fundamentos_Gestion_Datos)

#### Alineacion con el silabo

Este laboratorio desarrolla:

- Arquitectura DBMS con SQLite.
- DML: `INSERT`, `UPDATE`, `DELETE`.
- Transacciones: `BEGIN`, `COMMIT`, `ROLLBACK`.
- DCL conceptual: `GRANT`, `REVOKE`, roles y permisos.
- Almacenamiento local, nube e hibrido.

**Nota importante:** el laboratorio ahora detecta la tabla real disponible. Si tu base tiene `ventas_original`, trabajara con esa tabla. Si tiene `ventas`, trabajara con `ventas`. No exige tablas que tu base no tiene.

## ✓ Actividad 1 - Conceptos previos

| Concepto               | Respuesta |
|------------------------|-----------|
| DML                    |           |
| INSERT                 |           |
| UPDATE                 |           |
| DELETE                 |           |
| Transaccion            |           |
| COMMIT                 |           |
| ROLLBACK               |           |
| DCL                    |           |
| GRANT                  |           |
| REVOKE                 |           |
| Almacenamiento local   |           |
| Almacenamiento en nube |           |
| Almacenamiento hibrido |           |

# Paso 1: librerias y configuracion para Colab

```
import shutil
import sqlite3
import urllib.request
from pathlib import Path
```

```
import pandas as pd
```

```
pd.set_option("display.max_columns", 100)
pd.set_option("display.max_rows", 80)
```

```
DB_URL = "https://raw.githubusercontent.com/Rociosayan/PMD1_Fundamentos_Gestion_Datos/main/casos/16_abarrotes_ventas_inventariar
DB_ORIGINAL = Path("abarrotes_ventas_inventario.db")
DB_TRABAJO = Path("trabajo_lab_d9.db")
```

```
print("Entorno listo")
```

Entorno listo

# Paso 2: descargar la base desde GitHub o subirla manualmente

```
def preparar_base():
    if not DB_ORIGINAL.exists() or DB_ORIGINAL.stat().st_size <= 1000:
        try:
            print("Descargando base SQLite desde GitHub...")
```

```

urllib.request.urlretrieve(DB_URL, DB_ORIGINAL)
if DB_ORIGINAL.stat().st_size <= 1000:
    raise ValueError("El archivo descargado no parece una base SQLite valida.")
print("Descarga correcta:", DB_ORIGINAL.resolve())
except Exception as error:
    print("No se pudo descargar automaticamente.")
    print("Detalle:", error)
    print("Sube manualmente el archivo .db que estas usando en el curso.")
from google.colab import files
uploaded = files.upload()
if not uploaded:
    raise FileNotFoundError("No se subio ningun archivo.")
nombre = next(iter(uploaded.keys()))
shutil.move(nombre, DB_ORIGINAL)

shutil.copy2(DB_ORIGINAL, DB_TRABAJO)
print("Copia de trabajo:", DB_TRABAJO.resolve())
print("Tamano:", DB_TRABAJO.stat().st_size, "bytes")

preparar_base()

```

Descargando base SQLite desde GitHub...  
 Descarga correcta: /content/abarrotes\_ventas\_inventario.db  
 Copia de trabajo: /content/trabajo\_lab\_d9.db  
 Tamano: 131072 bytes

```

# Paso 3: conectar e inspeccionar tablas reales

conn = sqlite3.connect(DB_TRABAJO)
conn.isolation_level = None
cur = conn.cursor()

tablas = pd.read_sql_query(
    '''
    SELECT name AS tabla
    FROM sqlite_master
    WHERE type = 'table'
    ORDER BY name;
    ''',
    conn
)

display(tablas)

if tablas.empty:
    raise ValueError("La base no contiene tablas.")

# Detectar tabla base real.
preferencias = ["ventas_original", "ventas", "dataset_abarrotes_limpio"]
tablas_disponibles = tablas["tabla"].tolist()

TABLA_BASE = None
for candidata in preferencias:
    if candidata in tablas_disponibles:
        TABLA_BASE = candidata
        break

if TABLA_BASE is None:
    TABLA_BASE = tablas_disponibles[0]

print("Tabla base seleccionada:", TABLA_BASE)
display(pd.read_sql_query(f"PRAGMA table_info('{TABLA_BASE}']", conn))
display(pd.read_sql_query(f"SELECT * FROM '{TABLA_BASE}' LIMIT 5", conn))

```

tabla

0 ventas\_original

Tabla base seleccionada: ventas\_original

|    | cid | name                     | type    | notnull | dflt_value | pk |
|----|-----|--------------------------|---------|---------|------------|----|
| 0  | 0   | id_venta                 | INTEGER | 0       | None       | 1  |
| 1  | 1   | fecha_operacion          | TEXT    | 0       | None       | 0  |
| 2  | 2   | cantidad_unidades        | TEXT    | 0       | None       | 0  |
| 3  | 3   | precio_venta_unitario    | TEXT    | 0       | None       | 0  |
| 4  | 4   | costo_unitario           | TEXT    | 0       | None       | 0  |
| 5  | 5   | descuento_pct            | TEXT    | 0       | None       | 0  |
| 6  | 6   | stock_inicial            | TEXT    | 0       | None       | 0  |
| 7  | 7   | stock_final              | TEXT    | 0       | None       | 0  |
| 8  | 8   | merma_unidades           | TEXT    | 0       | None       | 0  |
| 9  | 9   | canal                    | TEXT    | 0       | None       | 0  |
| 10 | 10  | metodo_pago              | TEXT    | 0       | None       | 0  |
| 11 | 11  | monto_venta_soles        | TEXT    | 0       | None       | 0  |
| 12 | 12  | margen_venta_soles       | TEXT    | 0       | None       | 0  |
| 13 | 13  | observacion              | TEXT    | 0       | None       | 0  |
| 14 | 14  | tiendas_nombre           | TEXT    | 0       | None       | 0  |
| 15 | 15  | tiendas_distrito         | TEXT    | 0       | None       | 0  |
| 16 | 16  | tiendas_zona             | TEXT    | 0       | None       | 0  |
| 17 | 17  | tiendas_tipo_local       | TEXT    | 0       | None       | 0  |
| 18 | 18  | productos_nombre         | TEXT    | 0       | None       | 0  |
| 19 | 19  | productos_categoria      | TEXT    | 0       | None       | 0  |
| 20 | 20  | productos_proveedor      | TEXT    | 0       | None       | 0  |
| 21 | 21  | productos_costo_unitario | REAL    | 0       | None       | 0  |
| 22 | 22  | productos_precio_lista   | REAL    | 0       | None       | 0  |
| 23 | 23  | clientes_nombre          | TEXT    | 0       | None       | 0  |
| 24 | 24  | clientes_distrito        | TEXT    | 0       | None       | 0  |
| 25 | 25  | clientes_segmento        | TEXT    | 0       | None       | 0  |
| 26 | 26  | clientes_fecha_registro  | TEXT    | 0       | None       | 0  |

|   | id_venta | fecha_operacion | cantidad_unidades | precio_venta_unitario | costo_unitario | descuento_pct | stock_inicial | stock_fin |
|---|----------|-----------------|-------------------|-----------------------|----------------|---------------|---------------|-----------|
| 0 | 1        | 2026-01-22      | 67                | 15.06                 | 10.04          | 8             | 82            |           |
| 1 | 2        | 2026-01-04      | 37                | 9.42                  | 5.94           | 8 aprox       | 223           | 1         |
| 2 | 3        | 2026-02-13      | 8                 | 5.44                  | 3.46           | 0             | 98            |           |
| 3 | 4        | 06/06/2026      | 44                | 12.76                 | 8.24           | 3             | 167           | 1         |
| 4 | 5        | 2026-02-19      | 61                | 9.28                  | 5.96           | 0             | 180           | 1         |

▼ Pregunta 1

Que tabla real detecto el notebook? Por que eso evita trabajar con una base ficticia?

Respuesta: \_\_\_\_\_

```
# Paso 4: crear tabla de trabajo estandarizada desde datos reales

df_base = pd.read_sql_query(f"SELECT * FROM '{TABLA_BASE}'", conn)

def buscar_columna(posibles):
    columnas = list(df_base.columns)
    columnas_lower = {c.lower(): c for c in columnas}
    for posible in posibles:
        posible = posible.lower()
        if posible in columnas_lower:
            return columnas_lower[posible]
    for c in columnas:
        c_lower = c.lower()
        if any(posible.lower() in c_lower for posible in posibles):
            return c
    return None

def limpiar_numero(serie):
    return (
        serie.astype(str)
        .str.replace("S/", "", regex=False)
        .str.replace(",", ".", regex=False)
        .str.extract(r"([+-]?[0-9]*\.[0-9]+)", expand=False)
        .astype(float)
    )

col_id = buscar_columna(["id_venta", "id", "codigo"])
col_fecha = buscar_columna(["fecha_operacion", "fecha", "date"])
col_producto = buscar_columna(["producto", "nombre_producto", "item"])
col_cantidad = buscar_columna(["cantidad_unidades", "cantidad", "unidades"])
col_monto = buscar_columna(["monto_venta_soles", "monto", "total", "venta"])

df_lab = pd.DataFrame()
df_lab["id_operacion"] = range(1, len(df_base) + 1)
df_lab["fuente_tabla"] = TABLA_BASE
df_lab["id_origen"] = df_base[col_id].astype(str) if col_id else df_lab["id_operacion"].astype(str)
df_lab["fecha_operacion"] = df_base[col_fecha].astype(str) if col_fecha else ""
df_lab["producto"] = df_base[col_producto].astype(str) if col_producto else "registro_real"
df_lab["cantidad"] = limpiar_numero(df_base[col_cantidad]).fillna(0) if col_cantidad else 0
df_lab["monto"] = limpiar_numero(df_base[col_monto]).fillna(0) if col_monto else 0
df_lab["estado"] = "ORIGINAL"
df_lab["observacion"] = "BASE_REAL"

cur.execute("DROP TABLE IF EXISTS ventas_lab_d9")
df_lab.to_sql("ventas_lab_d9", conn, if_exists="replace", index=False)

print("Columnas detectadas:")
print("ID:", col_id, "| Fecha:", col_fecha, "| Producto:", col_producto, "| Cantidad:", col_cantidad, "| Monto:", col_monto)
print("Tabla de trabajo creada: ventas_lab_d9")
display(pd.read_sql_query("SELECT * FROM ventas_lab_d9 LIMIT 10", conn))
```

Columnas detectadas:  
ID: id\_venta | Fecha: fecha\_operacion | Producto: productos\_nombre | Cantidad: cantidad\_unidades | Monto: monto\_venta\_soles  
Tabla de trabajo creada: ventas\_lab\_d9

|   | id_operacion | fuente_tabla    | id_origen | fecha_operacion | producto          | cantidad | monto   | estado   | observacion |  |
|---|--------------|-----------------|-----------|-----------------|-------------------|----------|---------|----------|-------------|--|
| 0 | 1            | ventas_original | 1         | 2026-01-22      | Cafe instantaneo  | 67.0     | 1009.02 | ORIGINAL | BASE_REAL   |  |
| 1 | 2            | ventas_original | 2         | 2026-01-04      | Papel higienico   | 37.0     | 348.54  | ORIGINAL | BASE_REAL   |  |
| 2 | 3            | ventas_original | 3         | 2026-02-13      | Lenteja 500g      | 8.0      | 43.52   | ORIGINAL | BASE_REAL   |  |
| 3 | 4            | ventas_original | 4         | 06/06/2026      | Aceite vegetal 1L | 44.0     | 561.44  | ORIGINAL | BASE_REAL   |  |
| 4 | 5            | ventas_original | 5         | 2026-02-19      | Papel higienico   | 61.0     | 566.08  | ORIGINAL | BASE_REAL   |  |
| 5 | 6            | ventas_original | 6         | 2026-01-09      | Arroz extra 5kg   | 21.0     | 607.11  | ORIGINAL | BASE_REAL   |  |
| 6 | 7            | ventas_original | 7         | 2026-01-04      | Yogurt familiar   | 55.0     | 394.90  | ORIGINAL | BASE_REAL   |  |
| 7 | 8            | ventas_original | 8         | 2026-01-13      | galletas soda     | 51.0     | 140.76  | ORIGINAL | BASE_REAL   |  |
| 8 | 9            | ventas_original | 9         | 2026-03-21      | Papel higienico   | 48.0     | 446.88  | ORIGINAL | BASE_REAL   |  |
| 9 | 10           | ventas_original | 10        | 2026-02-13      | Lenteja 500g      | 8.0      | 43.52   | ORIGINAL | BASE_REAL   |  |

Actividad 2 - DML con transacciones

Las operaciones se hacen sobre `ventas_lab_d9`, una tabla de trabajo creada a partir de los registros reales detectados en la base. Esto protege la tabla original y permite practicar DML sin dañar datos fuente.

# Paso 5: evidencia inicial

```
display(pd.read_sql_query("SELECT COUNT(*) AS registros_iniciales FROM ventas_lab_d9", conn))
display(pd.read_sql_query("SELECT estado, observacion, COUNT(*) AS registros FROM ventas_lab_d9 GROUP BY estado, observacion",
```

| registros_iniciales |  |     |
|---------------------|--|-----|
| 0                   |  | 500 |

  

|   | estado   | observacion | registros |
|---|----------|-------------|-----------|
| 0 | ORIGINAL | BASE_REAL   | 500       |

# Paso 6: INSERT de 10 registros basados en registros reales

```
base_insert = pd.read_sql_query(
    "SELECT fecha_operacion, producto, cantidad, monto FROM ventas_lab_d9 WHERE estado = 'ORIGINAL' LIMIT 10",
    conn
)

max_id = int(pd.read_sql_query("SELECT MAX(id_operacion) AS max_id FROM ventas_lab_d9", conn).iloc[0]["max_id"])

nuevos = []
for i, row in base_insert.iterrows():
    nuevos.append((
        max_id + i + 1,
        TABLA_BASE,
        f"COPIA_REAL_{i+1}",
        row["fecha_operacion"],
        row["producto"],
        float(row["cantidad"]),
        float(row["monto"]),
        "INSERTADO",
        "LAB_D9_INSERT"
    ))

try:
    cur.execute("BEGIN")
    cur.executemany(
        '''
        INSERT INTO ventas_lab_d9 (
            id_operacion, fuente_tabla, id_origen, fecha_operacion,
            producto, cantidad, monto, estado, observacion
        )
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        ''',
        nuevos
    )
    cur.execute("COMMIT")
    print("INSERT confirmado:", len(nuevos), "registros")
except Exception as error:
    cur.execute("ROLLBACK")
    raise RuntimeError(f"Error en INSERT: {error}")

display(pd.read_sql_query("SELECT * FROM ventas_lab_d9 WHERE observacion = 'LAB_D9_INSERT'", conn))
```

INSERT confirmado: 10 registros

|   | id_operacion | fuentes_tabla   | id_origen    | fecha_operacion | producto          | cantidad | monto   | estado    | observacion   |  |
|---|--------------|-----------------|--------------|-----------------|-------------------|----------|---------|-----------|---------------|--|
| 0 | 501          | ventas_original | COPIA_REAL_1 | 2026-01-22      | Cafe instantaneo  | 67.0     | 1009.02 | INSERTADO | LAB_D9_INSERT |  |
| 1 | 502          | ventas_original | COPIA_REAL_2 | 2026-01-04      | Papel higienico   | 37.0     | 348.54  | INSERTADO | LAB_D9_INSERT |  |
| 2 | 503          | ventas_original | COPIA_REAL_3 | 2026-02-13      | Lenteja 500g      | 8.0      | 43.52   | INSERTADO | LAB_D9_INSERT |  |
| 3 | 504          | ventas_original | COPIA_REAL_4 | 06/06/2026      | Aceite vegetal 1L | 44.0     | 561.44  | INSERTADO | LAB_D9_INSERT |  |
| 4 | 505          | ventas_original | COPIA_REAL_5 | 2026-02-19      | Papel higienico   | 61.0     | 566.08  | INSERTADO | LAB_D9_INSERT |  |
| 5 | 506          | ventas_original | COPIA_REAL_6 | 2026-01-09      | Arroz extra 5kg   | 21.0     | 607.11  | INSERTADO | LAB_D9_INSERT |  |
| 6 | 507          | ventas_original | COPIA_REAL_7 | 2026-01-09      | Yogurt            | 55.0     | 60.00   | INSERTADO | LAB_D9_INSERT |  |

## ▼ Pregunta 2

Por que el INSERT se hace dentro de una transaccion?

**Respuesta:** \_\_\_\_\_

```
# Paso 7: UPDATE de 3 registros

ids_update = pd.read_sql_query(
    "SELECT id_operacion FROM ventas_lab_d9 WHERE observacion = 'LAB_D9_INSERT' ORDER BY id_operacion LIMIT 3",
    conn
)["id_operacion"].tolist()

display(pd.read_sql_query(
    "SELECT * FROM ventas_lab_d9 WHERE id_operacion IN (?, ?, ?)",
    conn,
    params=ids_update
))

try:
    cur.execute("BEGIN")
    cur.executemany(
        '''
        UPDATE ventas_lab_d9
        SET estado = ?,
            observacion = ?,
            monto = ROUND(monto * 1.05, 2)
        WHERE id_operacion = ?
        ''',
        [("ACTUALIZADO", "LAB_D9_UPDATE", int(i)) for i in ids_update]
    )
    cur.execute("COMMIT")
    print("UPDATE confirmado:", len(ids_update), "registros")
except Exception as error:
    cur.execute("ROLLBACK")
    raise RuntimeError(f"Error en UPDATE: {error}")

display(pd.read_sql_query(
    "SELECT * FROM ventas_lab_d9 WHERE id_operacion IN (?, ?, ?)",
    conn,
    params=ids_update
))
```

|                                | id_operacion | fuentes_tabla   | id_origen    | fecha_operacion | producto         | cantidad | monto   | estado      | observacion   |
|--------------------------------|--------------|-----------------|--------------|-----------------|------------------|----------|---------|-------------|---------------|
| 0                              | 501          | ventas_original | COPIA_REAL_1 | 2026-01-22      | Cafe instantaneo | 67.0     | 1009.02 | INSERTADO   | LAB_D9_INSERT |
| 1                              | 502          | ventas_original | COPIA_REAL_2 | 2026-01-04      | Papel higienico  | 37.0     | 348.54  | INSERTADO   | LAB_D9_INSERT |
| 2                              | 503          | ventas_original | COPIA_REAL_3 | 2026-02-13      | Lenteja 500g     | 8.0      | 43.52   | INSERTADO   | LAB_D9_INSERT |
| UPDATE confirmado: 3 registros |              |                 |              |                 |                  |          |         |             |               |
|                                | id_operacion | fuentes_tabla   | id_origen    | fecha_operacion | producto         | cantidad | monto   | estado      | observacion   |
| 0                              | 501          | ventas_original | COPIA_REAL_1 | 2026-01-22      | Cafe instantaneo | 67.0     | 1059.47 | ACTUALIZADO | LAB_D9_UPDATE |

❯ Pregunta 3

Que riesgo existe al ejecutar un UPDATE sin WHERE?

Respuesta: \_\_\_\_\_

```
# Paso 8: DELETE de 2 registros con verificacion previa

ids_delete = pd.read_sql_query(
    "SELECT id_operacion FROM ventas_lab_d9 WHERE observacion = 'LAB_D9_INSERT' ORDER BY id_operacion DESC LIMIT 2",
    conn
)["id_operacion"].tolist()

print("Registros a eliminar:", ids_delete)
display(pd.read_sql_query(
    "SELECT * FROM ventas_lab_d9 WHERE id_operacion IN (?, ?)",
    conn,
    params=ids_delete
))

try:
    cur.execute("BEGIN")
    cur.executemany(
        "DELETE FROM ventas_lab_d9 WHERE id_operacion = ?",
        [(int(i),) for i in ids_delete]
    )
    cur.execute("COMMIT")
    print("DELETE confirmado:", len(ids_delete), "registros")
except Exception as error:
    cur.execute("ROLLBACK")
    raise RuntimeError(f"Error en DELETE: {error}")

display(pd.read_sql_query(
    "SELECT * FROM ventas_lab_d9 WHERE id_operacion IN (?, ?)",
    conn,
    params=ids_delete
))
```

Registros a eliminar: [510, 509]

|   | id_operacion | fuentes_tabla   | id_origen     | fecha_operacion | producto        | cantidad | monto  | estado    | observacion   |
|---|--------------|-----------------|---------------|-----------------|-----------------|----------|--------|-----------|---------------|
| 0 | 509          | ventas_original | COPIA_REAL_9  | 2026-03-21      | Papel higienico | 48.0     | 446.88 | INSERTADO | LAB_D9_INSERT |
| 1 | 510          | ventas_original | COPIA_REAL_10 | 2026-02-13      | Lenteja 500g    | 8.0      | 43.52  | INSERTADO | LAB_D9_INSERT |

DELETE confirmado: 2 registros

❯ Pregunta 4

Por que se debe verificar con SELECT antes de ejecutar DELETE?

Respuesta: \_\_\_\_\_

```
# Paso 9: demostracion de ROLLBACK

max_id = int(pd.read_sql_query("SELECT MAX(id_operacion) AS max_id FROM ventas_lab_d9", conn).iloc[0]["max_id"])
conteo_antes = int(pd.read_sql_query("SELECT COUNT(*) AS total FROM ventas_lab_d9", conn).iloc[0]["total"])
```

```
try:
    cur.execute("BEGIN")
    cur.execute(
        '''
        INSERT INTO ventas_lab_d9 (
            id_operacion, fuente_tabla, id_origen, fecha_operacion,
            producto, cantidad, monto, estado, observacion
        )
        VALUES (?, ?, ?, ?, ?, ?, ?, ?, ?)
        ''',
        (max_id + 1, TABLA_BASE, "ROLLBACK_TEST", "2026-06-30", "registro_prueba", 1, 10, "TEMPORAL", "LAB_D9_ROLLBACK")
    )
    print("Registro temporal insertado dentro de la transaccion.")
    raise Exception("Error simulado antes de COMMIT")
    cur.execute("COMMIT")
except Exception as error:
    cur.execute("ROLLBACK")
    print("ROLLBACK ejecutado:", error)

conteo_despues = int(pd.read_sql_query("SELECT COUNT(*) AS total FROM ventas_lab_d9", conn).iloc[0]["total"])
print("Conteo antes:", conteo_antes, "| Conteo despues:", conteo_despues)
display(pd.read_sql_query("SELECT * FROM ventas_lab_d9 WHERE observacion = 'LAB_D9_ROLLBACK'", conn))
```

Registro temporal insertado dentro de la transaccion.  
ROLLBACK ejecutado: Error simulado antes de COMMIT  
Conteo antes: 508 | Conteo despues: 508

| id_operacion | fuente_tabla | id_origen | fecha_operacion | producto | cantidad | monto | estado | observacion |
|--------------|--------------|-----------|-----------------|----------|----------|-------|--------|-------------|
|--------------|--------------|-----------|-----------------|----------|----------|-------|--------|-------------|



Pregunta 5

El registro temporal quedo guardado? Explica que hizo ROLLBACK.

Respuesta: \_\_\_\_\_

Actividad 3 - DCL conceptual

SQLite no implementa GRANT y REVOKE como motores cliente-servidor. Aun asi, el concepto de control de acceso es parte del silabo.

```
# Paso 10: matriz conceptual de roles

permisos = pd.DataFrame([
    ["analista_ventas", "SELECT", "No modifica registros"],
    ["operador_tienda", "SELECT, INSERT, UPDATE", "No elimina registros"],
    ["auditor", "SELECT", "Solo revision"],
    ["admin_bd", "ALL PRIVILEGES", "Uso restringido"]
], columns=["rol", "permisos", "restriccion"])

display(permisos)

print("Ejemplos conceptuales:")
print("GRANT SELECT ON ventas TO analista_ventas;")
print("GRANT INSERT, UPDATE ON ventas TO operador_tienda;")
print("REVOKE DELETE ON ventas FROM operador_tienda;")
```

|   | rol             | permisos               | restriccion           |
|---|-----------------|------------------------|-----------------------|
| 0 | analista_ventas | SELECT                 | No modifica registros |
| 1 | operador_tienda | SELECT, INSERT, UPDATE | No elimina registros  |
| 2 | auditor         | SELECT                 | Solo revision         |
| 3 | admin_bd        | ALL PRIVILEGES         | Uso restringido       |



Ejemplos conceptuales:  
GRANT SELECT ON ventas TO analista\_ventas;  
GRANT INSERT, UPDATE ON ventas TO operador\_tienda;  
REVOKE DELETE ON ventas FROM operador\_tienda;



Pregunta 6

Que riesgo existe si todos los usuarios pueden eliminar registros?

Respuesta: \_\_\_\_\_

Actividad 4 - Almacenamiento local, nube e hibrido

```
# Paso 11: comparacion de almacenamiento

almacenamiento = pd.DataFrame([
    ["Local", "SQLite en Colab/PC", "Rapido y simple", "Se pierde si no se respalda"],
    ["Nube", "GitHub, Drive, Cloud SQL", "Colaborativo y respaldado", "Requiere permisos"],
    ["Hibrido", "SQLite local + respaldo cloud", "Flexible", "Puede duplicar versiones"],
], columns=["tipo", "ejemplo", "ventaja", "riesgo"])

display(almacenamiento)
```

|   | tipo    | ejemplo                       | ventaja                   | riesgo                      |  |
|---|---------|-------------------------------|---------------------------|-----------------------------|--|
| 0 | Local   | SQLite en Colab/PC            | Rapido y simple           | Se pierde si no se respalda |  |
| 1 | Nube    | GitHub, Drive, Cloud SQL      | Colaborativo y respaldado | Requiere permisos           |  |
| 2 | Hibrido | SQLite local + respaldo cloud | Flexible                  | Puede duplicar versiones    |  |

Pasos siguientes: [Generar código con almacenamiento](#) [New interactive sheet](#)

```
# Paso 12: evidencia final

display(pd.read_sql_query(
    '''
        SELECT estado, observacion, COUNT(*) AS registros
        FROM ventas_lab_d9
        GROUP BY estado, observacion
        ORDER BY observacion, estado;
    ''',
    conn
))

display(pd.read_sql_query("SELECT COUNT(*) AS registros_rollback FROM ventas_lab_d9 WHERE observacion = 'LAB_D9_ROLLBACK'", conn))
display(pd.read_sql_query("SELECT COUNT(*) AS total_final FROM ventas_lab_d9", conn))

conn.close()
print("Conexion cerrada correctamente")
```

|                    | estado      | observacion   | registros |  |
|--------------------|-------------|---------------|-----------|--|
| 0                  | ORIGINAL    | BASE_REAL     | 500       |  |
| 1                  | INSERTADO   | LAB_D9_INSERT | 5         |  |
| 2                  | ACTUALIZADO | LAB_D9_UPDATE | 3         |  |
| registros_rollback |             |               |           |  |
| 0                  |             | 0             |           |  |
| total final        |             |               |           |  |